

使用 Dockerfile 開發容器

來源：<https://codelabs.developers.google.com/developing-containers-with-dockerfiles?hl=zh-tw>

步驟數：7

目錄

1. [1. 總覽](#)
2. [2. 範例應用程式](#)
3. [3. Dockerfile](#)
4. [4. 建構](#)
5. [5. 執行](#)
6. [6. 推送](#)
7. [7. 恭喜 !](#)

步驟 1 · 約 0 分鐘

1. 總覽

Docker 開放式平台可開發、推送及執行應用程式。有了 Docker，您可以將應用程式從基礎架構中獨立出來，並將基礎架構做為代管應用程式處理。這個平台可加速推送程式碼，並加快測試及部署作業，縮短從編寫到執行程式碼的週期。

為了實現這個目的，Docker 結合核心容器化功能，以及可協助您管理及部署應用程式的工作流程與工具。

Docker 容器可直接在 Kubernetes 中使用，能輕鬆在 Kubernetes Engine 內執行。學習 Docker 的基礎知識後，您就能開始開發 Kubernetes 與容器化應用程式。

課程內容

在本實驗室中，您將瞭解如何執行下列操作：

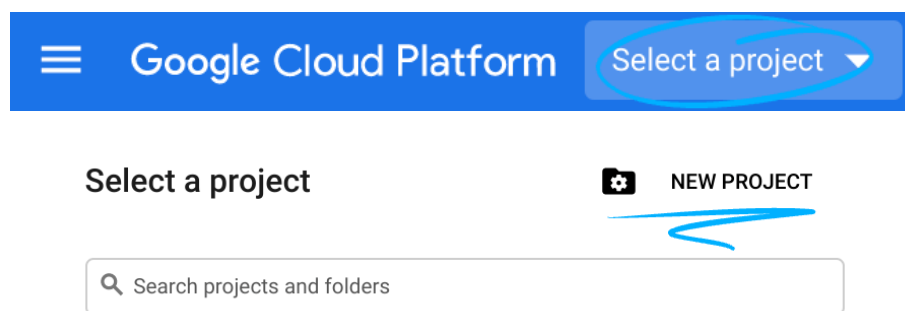
- 為範例應用程式建立 Dockerfile
- 建構映像檔
- 在本機將映像檔做為容器執行
- 變更容器行為
- 將映像檔推送至 Artifact Registry

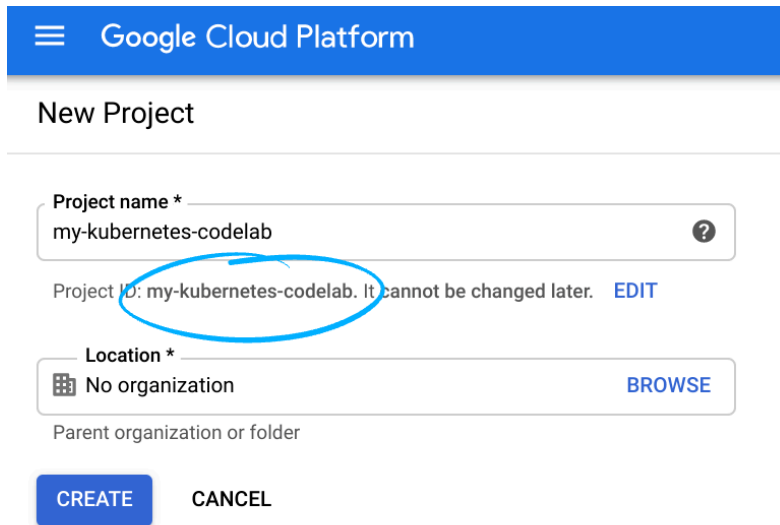
必要條件

本實驗室的難度為入門級。幾乎或完全沒有使用過 Docker 和容器，也可以進行本實驗室。熟悉 Cloud Shell 及指令列的使用方法會有幫助，但非必要。

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。





Google Cloud Platform

New Project

Project name *
my-kubernetes-codelab

Project ID: my-kubernetes-codelab. It cannot be changed later. [EDIT](#)

Location *
No organization [BROWSE](#)

Parent organization or folder

[CREATE](#) [CANCEL](#)

- 專案名稱是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串，您隨時可以更新。
- 專案 ID 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)，因此如果您不喜歡該字串，可以產生另一個隨機字串，或是嘗試使用自己的字串，看看是否可用。專案建立後，系統就會「凍結」該值。
- 還有第三個值，也就是部分 API 使用的「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。專案刪除後，該 ID 就無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

- 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要停用資源，避免在本教學課程結束後繼續產生帳單費用，請按照程式碼研究室結尾的「清除」操作說明操作。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

步驟 2 · 約 0 分鐘

2. 範例應用程式

我們提供範例應用程式，方便您完成本實驗室。在本節中，您將擷取原始碼，並以原生形式建構應用程式，然後再繼續進行容器化程序。

原始碼

本實驗室的原始碼位於 [GoogleCloudPlatform/container-developer-workshop](https://github.com/GoogleCloudPlatform/container-developer-workshop) 存放區，以及 [範例應用程式說明文件](#)。

設定 Git



```
git config --global user.name ${USER}
git config --global user.email ${USER}@qwiklabs.net
```

複製範例應用程式 Cloud Source Repositories

```
gcloud source repos clone sample-app ${HOME}/sample-app &&
cd ${HOME}/sample-app &&
git checkout main
```

輸出

```
Cloning into '/home/student_03_49720296e995/sample-app'...
remote: Finding sources: 100% (16/16)
remote: Total 16 (delta 0), reused 16 (delta 0)
Receiving objects: 100% (16/16), 47.23 KiB | 681.00 KiB/s, done.
warning: remote HEAD refers to nonexistent ref, unable to checkout.

Project [qwiklabs-gcp-02-4327c4e03d82] repository [sample-app] was cloned to
[/home/student_03_49720296e995/sample-app].
Branch 'main' set up to track remote branch 'main' from 'origin'.
Switched to a new branch 'main'
```

建構範例應用程式

```
cd ${HOME}/sample-app
./mvnw compile
```

輸出

```
[INFO] Scanning for projects...
...
[INFO] Compiling 1 source file to /home/student_03_49720296e995/sample-
app/target/classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.080 s
[INFO] Finished at: 2022-02-23T17:14:30Z
[INFO] -----
```

執行範例應用程式

```
cd ${HOME}/sample-app
./mvnw exec:java
```

輸出

```
[INFO] Scanning for projects...
...
Listening at http://localhost:8080
```

預覽執行中的應用程式

- 按一下 Cloud Shell 的「網頁預覽」按鈕
- 按一下「透過以下通訊埠預覽：8080」

後續步驟

- 在 Cloud Shell 中按下 CTRL + c 鍵，停止執行中的應用程式
-

3. Dockerfile

使用 Dockerfile 將應用程式容器化

如要將應用程式封裝到容器中，其中一種方法是使用 Dockerfile。Dockerfile 類似於指令碼，會指示 Daemon 如何組裝容器映像檔。詳情請參閱 [Dockerfile 參考文件](#)。

在範例應用程式存放區中建立空白的 Dockerfile。

```
touch ${HOME}/sample-app/Dockerfile
```

在您選擇的編輯器中開啟 Dockerfile。

```
vi ${HOME}/sample-app/Dockerfile
```

選擇起始圖片

如要使用 Dockerfile 方法建構容器，您必須直接瞭解應用程式，才能組裝容器。建立 Dockerfile 的第一步是選取要當做映像檔基礎的映像檔。這個映像檔應是由可信來源 (通常是貴公司) 維護及發布的父項或基礎映像檔。

`FROM` 指令會初始化新的建構階段，並為後續的循序指令設定基本映像檔。因此，`FROM` 指令通常是 Dockerfile 中的第一個指令，且只能在選用的 [ARG 指令](#) 之前，用來支援變數。

語法：`FROM <image>[:<tag> | @<digest>] [AS <name>]`

圖片格式為 `<image>:<tag>` 或 `<image>@<digest>`。如未指定標記或摘要，則預設為 `:latest` 標記。`<image>` 的格式會因用於儲存映像檔的登錄檔而異。如果是 Artifact Registry，`<image>` 格式為 `<region>-docker.pkg.dev/<project ID>/<repository name>/<image name>:<image tag>`。

在本實驗室中，我們使用公開的 `openjdk:11.0-jdk` 映像檔，請在 Dockerfile 中加入下列程式碼：

```
FROM openjdk:11.0-jdk
```

設定工作目錄

`WORKDIR` 指令會為 Dockerfile 中後續的任何循序指令設定工作目錄。詳情請參閱 [Dockerfile 參考文件](#) 的 `WORKDIR` 一節。

語法：`WORKDIR <path>`

在本實驗室中，我們使用 `/app` 目錄做為 `WORKDIR`，請在 Dockerfile 底部新增下列程式碼：

```
WORKDIR /app
```

複製應用程式檔案

`COPY` 指令會將目錄或檔案從 `<source>` 位置複製到映像檔檔案系統的 `<destination>` 路徑。您可以指定多個 `<source>` 資源，這些資源都與建構環境相關。我們會在「建構」一節中進一步說明建構環境。詳情請參閱 [Dockerfile 參考文件](#) 的 `COPY` 區段。

語法：`COPY <source>... <destination>`

在本實驗室中，我們會將存放區中的所有檔案複製到映像檔檔案系統，並在 Dockerfile 底部新增下列程式碼行：

```
COPY . /app
```

編譯應用程式

RUN 指令會在目前圖片上方的全新圖片圖層中執行指令，並提交結果。產生的已提交映像檔會用於 Dockerfile 中的後續步驟。詳情請參閱 [Dockerfile 參考說明文件的 RUN 一節](#)。

語法：**RUN** <command>

在本實驗室中，我們將使用 Maven 將應用程式編譯為 JAR 檔案，並在 Dockerfile 底部新增下列程式碼

```
RUN ./mvnw compile assembly:single
```

開始申請

CMD 指令會為執行中的容器提供預設指令。Dockerfile 中只能有一個 CMD 指令，如果指定多個 CMD，只有最後一個 CMD 會生效。使用 CMD 和 ENTRYPOINT 指令可取得更多進階功能，但本實驗室不會介紹這些功能。詳情請參閱 [Dockerfile 參考說明文件中的「CMD」一節](#)。

語法：**CMD** ["executable","param1","param2"]

在本實驗室中，我們將執行編譯的 JAR 檔案，並在 Dockerfile 底部新增下列程式碼：

```
CMD ["java","-jar","/app/target/sample-app-1.0.0-jar-with-dependencies.jar"]
```

最終 Dockerfile

最終的 Dockerfile 會是

```
FROM openjdk:11.0-jdk
WORKDIR /app
COPY . /app
RUN ./mvnw compile assembly:single
CMD ["java","-jar","/app/target/sample-app-1.0.0-jar-with-dependencies.jar"]
```

在本機提交 Dockerfile

```
cd ${HOME}/sample-app
git add Dockerfile
git commit -m "Added Dockerfile"
```

步驟 4 · 約 0 分鐘

4. 建構

現在，我們將使用 `docker build` 指令，從 Dockerfile 建構映像檔。這個指令會指示 Docker Daemon 使用 Dockerfile 中的指令建構映像檔。詳情請參閱 [docker build 參考文件](#)。

建構映像檔

```
cd ${HOME}/sample-app
export IMAGE_TAG=$(git rev-parse --short HEAD)
docker build --tag sample-app:${IMAGE_TAG} .
```

輸出

```
Sending build context to Docker daemon 221.2kB
Step 1/4 : FROM openjdk:11.0-jdk
11.0-jdk: Pulling from library/openjdk
0c6b8ff8c37e: Pull complete
412caad352a3: Pull complete
e6d3e61f7a50: Pull complete
461bb1d8c517: Pull complete
e442ee9d8dd9: Pull complete
542c9fe4a7ba: Pull complete
41de18d1833d: Pull complete
Digest: sha256:d72b1b9e94e07278649d91c635e34737ae8f181c191b771bde6816f9bb4bd08a
Status: Downloaded newer image for openjdk:11.0-jdk
---> 2924126f1829
Step 2/4 : WORKDIR /app
---> Running in ea037abb273d
Removing intermediate container ea037abb273d
---> bd9b6d078082
Step 3/4 : COPY . /app
---> b9aec2b5de51
Step 4/4 : RUN ./mvnw compile jar:jar
---> Running in 3f5ff737b7fd
[INFO] Scanning for projects...
...
[INFO] Building jar: /app/target/sample-app-1.0.0.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.952 s
[INFO] Finished at: 2022-02-23T18:09:08Z
[INFO] -----
Removing intermediate container 331443caebd3
---> 152f65cc441e
Step 5/5 : CMD ["java", "-jar", "/app/target/sample-app-1.0.0.jar"]
---> Running in 3d595a72231c
Removing intermediate container 3d595a72231c
---> 0e40d7548cab
Successfully built 0e40d7548cab
Successfully tagged sample-app:aaa8895
```

步驟 5 · 約 0 分鐘

5. 執行

成功建構容器映像檔後，我們現在可以使用 `docker run` 指令執行應用程式，並確保應用程式的行為符合預期。這項指令會在命令提示字元的背景中啟動容器，以供測試或偵錯。詳情請參閱 [docker run 參考文件](#)。

使用映像檔執行容器

```
cd ${HOME}/sample-app
export IMAGE_TAG=$(git rev-parse --short HEAD)
docker run \
  --rm \
  -p 8080:8080 \
  sample-app:${IMAGE_TAG}
```

輸出

```
Listening at http://localhost:8080
```

預覽在容器中執行的應用程式

- 按一下 Cloud Shell 的「網頁預覽」按鈕
- 按一下「透過以下通訊埠預覽：8080」
- 在 Cloud Shell 中按下 `Ctrl + c` 鍵，停止容器

變更容器行為

執行 Docker Run 時，會使用 Dockerfile 中的預設設定。您可以新增其他指令和參數來修改這項行為。

啟用 TRACE 記錄功能

```
cd ${HOME}/sample-app
export IMAGE_TAG=$(git rev-parse --short HEAD)
docker run \
  --rm \
  -p 8080:8080 \
  sample-app:${IMAGE_TAG} \
  java -Dorg.slf4j.simpleLogger.defaultLogLevel=trace -jar /app/target/sample-app-1.0.0-jar-with-dependencies.jar
```

預覽在容器中執行的應用程式

- 按一下 Cloud Shell 的「網頁預覽」按鈕
- 按一下「透過以下通訊埠預覽：8080」
- 切換至 Cloud Shell 分頁，查看額外的記錄
- 在 Cloud Shell 中按下 `Ctrl + c` 鍵，停止容器。

變更通訊埠

```
cd ${HOME}/sample-app
export IMAGE_TAG=$(git rev-parse --short HEAD)
docker run \
--rm \
-e PORT=8081 \
-p 8081:8081 \
sample-app:${IMAGE_TAG}
```

預覽在容器中執行的應用程式

- 按一下 Cloud Shell 的「網頁預覽」按鈕
 - 按一下「變更通訊埠」
 - 輸入 8081
 - 按一下「變更並預覽」
 - 在 Cloud Shell 中按下 Ctrl + c 鍵，停止容器。
-

步驟 6 · 約 0 分鐘

6. 推送

確認容器映像檔正常運作後，如要讓其他環境和/或其他使用者執行這個容器，我們需要將映像檔推送至共用存放區。這應該是自動建構管道的一部分，但在測試環境中，我們已設定存放區，可以手動推送映像檔。

將 Dockerfile 提交至 sample-app 存放區

```
cd ${HOME}/sample-app
export IMAGE_TAG=$(git rev-parse --short HEAD)
git push
```

為 Artifact Registry 標記映像檔

```
docker tag sample-app:${IMAGE_TAG} \
  us-central1-docker.pkg.dev/${GOOGLE_CLOUD_PROJECT}/apps/sample-app:${IMAGE_TAG}
```

設定 Artifact Registry 的憑證

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

當系統提示時，請輸入 `Do you want to continue (Y/n)?` 回答 `y`，然後按下 `Enter`

將映像檔推送至 Artifact Registry

```
docker push us-central1-docker.pkg.dev/${GOOGLE_CLOUD_PROJECT}/apps/sample-app:${IMAGE_TAG}
```

輸出

```
The push refers to repository [us-central1-docker.pkg.dev/qwiklabs-gcp-04-
b47ced695a3c/apps/sample-app]
453b97f86449: Pushed
e86791aa0382: Pushed
d404c7ee0850: Pushed
fe4f44af763d: Pushed
7c072cee6a29: Pushed
1e5fdc3d671c: Pushed
613ab28cf833: Pushed
bed676ceab7a: Pushed
6398d5cccd2c: Pushed
0b0f2f2f5279: Pushed
aaa8895: digest:
sha256:459de00f86f159cc63f98687f7c9563fd65a2eb9bcc71c23dda3351baf13607a size: 2424
```

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容

- 為範例應用程式建立 Dockerfile
 - 建構映像檔
 - 在本機將映像檔做為容器執行
 - 容器行為變更
 - 將映像檔推送至 Artifact Registry
-